Niveau : 1ère année
Filière : IACS & G2ER
Année universitaire : 2024/2025

TP2 - Initiation à Python

Manipuler les variables, les opérateurs, les conditions, et les boucles de manière poussée.

Exercice 1 : Jeu de devinettes

Créez un programme où l'ordinateur génère un nombre aléatoire entre 1 et 100 (utilisez random.randint). Demandez à l'utilisateur de deviner ce nombre. À chaque tentative, indiquez si le nombre est plus grand ou plus petit.

Instructions *:*

- Affichez un message pour chaque tentative pour guider l'utilisateur.
- Limitez le nombre d'essais à 10. Si l'utilisateur ne devine pas, terminez le programme et affichez un message indiquant la réponse correcte.

Importation de la bibliothèque random

Dans cet exercice, vous allez utiliser des valeurs aléatoires pour rendre le jeu de devinettes plus intéressant. Pour cela, nous utiliserons le module random de Python, qui contient des fonctions permettant de générer des nombres aléatoires et de simuler des processus aléatoires.

Avant de commencer, vous devez **importer la bibliothèque random** en ajoutant la ligne suivante au début de votre code : « `import random` »

Cette ligne indique à Python de charger le module random, ce qui rend disponibles les fonctions qu'il contient, comme random.randint(), qui génère un nombre entier aléatoire entre deux valeurs spécifiées.

Dans cet exercice, nous utiliserons random.randint(1, 100) pour générer un nombre aléatoire entre 1 et 100, que l'utilisateur devra deviner.

N'oubliez pas d'inclure cette ligne d'importation pour que votre code fonctionne correctement.

Exercice 2 : Nombre Armstrong (ou narcissique)

Écrire un programme qui vérifie si un nombre de trois chiffres est un "nombre Armstrong" (ou narcissique). Un nombre est dit Armstrong si la somme de ses chiffres élevés à la puissance du nombre de chiffres est égale au nombre lui-même.

Exemple de sortie :

```
Entrez un nombre de trois chiffres : 153
153 est un nombre Armstrong.
```

Indication **:** 153 est un nombre Armstrong car $1^3+5^3+3^3=153$

Exercice 3 : Vérification de palindrome sans chaînes

Objectif : Écrire un programme qui vérifie si un nombre entier est un palindrome, sans convertir le nombre en chaîne de caractères.

Exercice 4 : Calcul de la racine carrée par dichotomie

Écrire un programme qui calcule la racine carrée entière d'un nombre n sans utiliser les fonctions mathématiques comme sqrt(). Utilisez une approche par dichotomie (recherche binaire).

Instructions :

1. Demandez à l'utilisateur d'entrer un nombre entier positif `n`.
2. Utilisez une méthode de recherche binaire pour trouver la plus grande valeur entière `x` telle que `x * x <= n`.
3. Affichez la racine carrée entière de `n`.

Exemple de sortie :

```
Entrez un nombre entier : 20
La racine carrée entière de 20 est : 4
```

Indication :

Initialisez `bas = 0` et `haut = n`. Utilisez une boucle pour diviser en deux l'intervalle jusqu'à trouver `x`. La recherche binaire est une méthode efficace pour calculer la racine carrée sans calculs directs de puissance.

Exercice 5 : Nombre parfait le plus proche

Écrire un programme qui trouve le nombre parfait le plus proche d'un nombre `n`, sans utiliser de structures de données.

Exemple de sortie :

```
Entrez un nombre entier : 20
Le nombre parfait le plus proche de 20 est : 28
```

Instructions pour le Compte Rendu

À la fin de ce TP, vous êtes invités à rédiger un compte rendu détaillé de vos travaux. Ce rapport doit inclure une analyse du code : expliquez le fonctionnement de votre code pour chaque exercice, tout en incluant des captures d'écran du code et de la sortie du code.

Critères d'évaluation :

- La clarté et la structure de votre rapport.
- La profondeur de votre analyse et votre capacité à justifier vos choix.
- La présentation générale (orthographe, grammaire, mise en page).

Chaque rapport bien rédigé et bien structuré sera pris en compte pour l'évaluation finale, et une note élevée sera attribuée aux travaux qui montrent un effort sérieux et une réflexion approfondie.